

Práctica 1

Primeros pasos JDBC (PostgreSQL)

1. Objetivos de la práctica

- Crear un repositorio Git y hacer commits desde IntelliJ.
- Crear una base de datos y una tabla simple en PostgreSQL.
- Añadir el driver JDBC de PostgreSQL en un proyecto Maven.
- Conectar Java a PostgreSQL (Connection).
- Ejecutar un SELECT y recorrer el ResultSet.
- Convertir filas a objetos (Libro) usando un DAO.

Al terminar la práctica, debes ser capaz de ejecutar un main que muestre por consola la lista de libros de la base de datos.

2. Checklist antes de empezar

Comprueba que tienes instalado:

- IntelliJ IDEA.
- JDK 21 (Ojo: versiones posteriores del JDK pueden dar problemas de incompatibilidad en alguna de las siguientes practicas con Spring Boot/JPA)
- PostgreSQL y pgAdmin.
- Git for Windows.

2.1 Comprobar Git en tu ordenador (Windows)

Abre PowerShell y ejecuta:

```
git --version
where git
```

PROBLEMA: Si IntelliJ dice “Cannot Run Git: git.exe not found”

Cierra IntelliJ y ábrelo de nuevo. Luego ve a File > Settings > Version Control > Git y pulsa “Test”. Si aún falla, busca manualmente la ruta a git.exe (normalmente en C:\Program Files\Git\cmd\git.exe).

3. Repaso Git

Git sirve para guardar el historial del proyecto. Cada guardado se llama commit. Piensa en un commit como una “foto” del proyecto.

En esta asignatura lo usaremos para:

- no perder trabajo si algo se rompe,
- volver a una versión que funcionaba,
- entregar prácticas con un historial claro,
- trabajar por partes (JDBC -> JPA -> Spring).

3.1 Tres conceptos que debes memorizar

- Working directory: tus archivos tal cual están en la carpeta.
- Staging area (Add): lo que preparas para el próximo commit.
- Commit: la “foto” guardada con un mensaje.

Primero preparo (add), luego guardo (commit). En IntelliJ esto se hace marcando archivos y pulsando Commit.

3.2 Configurar tu firma (solo una vez)

Git necesita tu nombre y correo para firmar los commits. En PowerShell:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tuemail@correo.com"

git config --global init.defaultBranch main

git config --global --list
```

3.3 Crear el repositorio en IntelliJ

1. En IntelliJ: VCS > Create Git Repository...
2. Selecciona la carpeta raíz del proyecto (la que contiene pom.xml).
3. Abre View > Tool Windows > Commit (Ctrl+K) para ver cambios.

3.4 .gitignore: qué es y por qué importa

El archivo .gitignore le dice a Git qué cosas NO debe guardar (por ejemplo: compilados, cachés del IDE). Normalmente, **Maven en IntelliJ crea por defecto ese archivo (debe encontrarse en la raíz de tu Proyecto, en el menú Izquierdo en IntelliJ)**. De no ser así deberás crear el archivo tu mismo:

En la **raíz del proyecto**:

1. Botón derecho sobre el proyecto → New > File
2. Nombre: .gitignore (con el punto delante, sin extensión)

Debes asegurarte que el contenido incluye dentro de target/ al menos:

```
.idea/  
*.iml  
*.log
```

OJO: No subas la carpeta .idea

La carpeta .idea contiene ajustes de IntelliJ específicos de TU ordenador. En repos compartidos suele dar problemas. Por eso la ignoramos con .gitignore.

3.5 Cómo hacer commits “por partes”

En la ventana Commit de IntelliJ puedes marcar/desmarcar archivos para incluir SOLO lo que toca en ese commit.

Ejemplo de commits (recomendado):

- Inicio proyecto (src + .gitignore)
- Añade driver JDBC PostgreSQL (configuración del pom.xml)
- Config conexión DB (Db.java + properties)
- SELECT libros (model+dao+app(main))

3.6 Si te equivocas con un commit

Estas opciones se usan mucho al principio (cuando todo es local y aún no hay push):

- Amend: reemplaza el último commit (para corregirlo).
- Undo Commit: deshace el commit pero mantiene los cambios en tu carpeta para volver a committear por partes.
- Revert: crea un commit nuevo que deshace otro commit (útil cuando ya está publicado).

4. PostgreSQL: base de datos de trabajo

Hoy trabajaremos con una base de datos de ejemplo y una tabla simple. La idea es practicar el flujo JDBC sin tratar aún relaciones.

Nombre sugerido: biblioteca_ad (todo en minúsculas).

4.1 Crear la base de datos

En pgAdmin ejecuta:

```
CREATE DATABASE biblioteca_ad;
```

4.2 Crear tabla y datos de ejemplo

Conéctate a la base de datos biblioteca_ad y ejecuta este script:

```
CREATE TABLE IF NOT EXISTS libro (  
    id SERIAL PRIMARY KEY,  
    titulo VARCHAR(200) NOT NULL,  
    isbn VARCHAR(20) UNIQUE,  
    anio INT,  
    disponible BOOLEAN NOT NULL DEFAULT true  
);  
  
TRUNCATE TABLE libro RESTART IDENTITY;  
  
INSERT INTO libro (titulo, isbn, anio, disponible) VALUES  
('Harry Potter y la Piedra filosofal', '9780132350884', 1999, true),  
('Los Pilares de la Tierra', '9780134685991', 1998, true),  
('El Hobbit', '9780201633610', 1964, false);  
  
SELECT * FROM libro ORDER BY id;
```

Explicación rápida de lo que hace:

- CREATE TABLE: crea la tabla si no existe.
- SERIAL: entero autoincremental (id).
- TRUNCATE ... RESTART IDENTITY: limpia la tabla y reinicia el contador del id (para repetir prácticas).
- INSERT: mete 3 libros de ejemplo.
- SELECT: comprueba que hay datos y los muestra.

4.3 Usuario en pgadmin ¿cómo saberlo?

En la barra superior de pgAdmin suele aparecer algo parecido a: biblioteca_ad/postgres@PostgreSQL 18.

Eso significa:

- Nombre de la base de datos: biblioteca_ad
- Usuario: postgres

Para verificar el usuario actual, en Query Tool:

```
SELECT current_user;
```

5. Proyecto Maven en IntelliJ

Maven organiza proyectos Java de forma estándar y descarga dependencias automáticamente. Aquí es donde añadiremos el driver JDBC.

5.1 groupId y artifactId

En Maven, un proyecto se identifica por: groupId + artifactId + version.

- groupId: quién “firma” el proyecto (organización). En clase puede ser com.dam.ad.
- artifactId: nombre del proyecto (por ejemplo biblioteca-jdbc).
- version: versión del proyecto (1.0-SNAPSHOT al empezar).

5.2 Estructura típica de carpetas

- src/main/java: código Java.
- src/main/resources: ficheros de configuración (por ejemplo db.properties).
- pom.xml: el archivo de configuración de Maven (dependencias).
- target/: salida de compilación (se ignora en Git).

6. JDBC paso a paso

JDBC es la API estándar de Java para hablar con bases de datos relacionales.

El flujo típico es siempre este: Driver -> Connection -> PreparedStatement -> ResultSet -> cerrar recursos.

6.1 Añadir el driver JDBC de PostgreSQL

En pom.xml, dentro de <dependencies>, añade:

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.7.3</version>
</dependency>
```

Después, recarga Maven (Reload). Sin esto, Java no sabe hablar con PostgreSQL.

6.2 Crear db.properties (configuración)

Crea src/main/resources/db.properties con TU configuración real:

```
db.url=jdbc:postgresql://localhost:5432/biblioteca_ad
db.user=postgres
db.password=postgres
```

OJO: no commitear contraseñas

En proyectos reales (y también en clase), no se sube db.properties real al repositorio porque contiene credenciales. Se puede subir un archivo de ejemplo (db.properties.example) y luego ya cada uno crea su db.properties local.

Recomendación para clase:

- Crea también src/main/resources/db.properties.example (sin password real).
- Añade src/main/resources/db.properties al .gitignore.

Línea a añadir en .gitignore

```
src/main/resources/db.properties
```

Si lo commiteaste por error y quieres sacarlo del repositorio sin borrarlo de tu PC:

```
git rm --cached src/main/resources/db.properties
```

6.3 Clase Db: obtener una Connection

Crea un paquete db y luego una clase Db que sera la encargada de centralizar la lectura de properties y la creación de la conexión con la base de datos.

```
package com.dam.ad.db;

import java.io.InputStream;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;

public class Db {
    private static final Properties PROPS = new Properties();

    static {
        try (InputStream in =
            Db.class.getClassLoader().getResourceAsStream("db.properties")) {
            if (in == null) {
                throw new RuntimeException("No se encuentra db.properties en
src/main/resources");
            }
            PROPS.load(in);
        } catch (Exception e) {
            throw new RuntimeException("Error cargando db.properties", e);
        }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(
            PROPS.getProperty("db.url"),
            PROPS.getProperty("db.user"),
            PROPS.getProperty("db.password")
        );
    }
}
```

1. **Al arrancar el programa**, cargamos las propiedades de db.properties en memoria:
 - `import java.io.InputStream;`
Importa `InputStream`, que es el tipo que se usa para leer un archivo como flujo de bytes. Aquí se usa para leer db.properties desde src/main/resources.
 - `import java.sql.Connection;`
Importa `Connection`, que representa una conexión abierta a la base de datos.
 - `import java.sql.DriverManager;`
Importa `DriverManager`, que es la clase que “fabrica” conexiones JDBC cuando le das una URL, usuario y contraseña.
 - `import java.sql.SQLException;`
Importa la excepción típica de JDBC: se lanza si falla la conexión o algo relacionado con SQL.

- `import java.util.Properties;`
Importa `Properties`, que es un “mapa” clave-valor pensado justo para ficheros `.properties` (`db.url=...`, etc.).
-

- `private static final Properties PROPS = new Properties();`
Crea un objeto `Properties` llamado `PROPS`.
 - `static` -> existe **una sola copia** para toda la aplicación.
 - `final` -> la referencia no cambia (siempre usaremos el mismo `PROPS`).
Aquí se guardarán las claves `db.url`, `db.user`, `db.password`.
-

- `static { ... }`
Esto es un **bloque estático**: se ejecuta **una sola vez**, automáticamente, cuando la clase `Db` se carga por primera vez (antes de usar `getConnection()`).
 - `try (InputStream in = Db.class.getClassLoader().getResourceAsStream("db.properties")) {`
Intenta abrir el archivo `db.properties` como un `InputStream`.
 - `getClassLoader().getResourceAsStream(...)` busca el archivo dentro del **classpath** (o sea, dentro de `src/main/resources`).
 - El `try-with-resources` garantiza que el `InputStream` se cierra solo al terminar (aunque haya error).
 - `if (in == null) { throw new RuntimeException("No se encuentra db.properties en src/main/resources"); }`
Si `in` es `null`, significa que el archivo **no se encontró**. Entonces se lanza una excepción clara para que el error sea fácil de entender (en vez de petar más tarde con un `NPE`).
 - `PROPS.load(in);`
Lee el contenido del archivo y lo mete dentro del objeto `PROPS`.
Ejemplo: si en el archivo pone `db.url=jdbc:postgresql://...`, entonces `PROPS` guardará esa pareja clave-valor.
 - `} catch (Exception e) { throw new RuntimeException("Error cargando db.properties", e); }`
Si ocurre cualquier error leyendo el archivo (no existe, está mal, permisos, etc.), se lanza una `RuntimeException` con un mensaje claro y el error original dentro (`e`) para poder ver la causa.
-

2. `getConnection()`, crea y devuelve una **Connection** usando esos datos:

- `public static Connection getConnection() throws SQLException {`
Método público y estático que devuelve una `Connection`.
- `return DriverManager.getConnection(PROPS.getProperty("db.url"),`
`PROPS.getProperty("db.user"), PROPS.getProperty("db.password"));`
 1. `PROPS.getProperty("db.url")` saca la URL JDBC del fichero.
 2. `PROPS.getProperty("db.user")` saca el usuario.
 3. `PROPS.getProperty("db.password")` saca la contraseña.

6.4 Modelo Libro

Crea un paquete `model` y luego una clase `Libro`. Cada fila de la tabla `libro` de la base de datos la representamos con un objeto `Libro`.

```
package com.dam.ad.model;

public class Libro {
    private final int id;
    private final String titulo;
    private final String isbn;
    private final Integer anio;
    private final boolean disponible;

    public Libro(int id, String titulo, String isbn, Integer anio, boolean
disponible) {
        this.id = id;
        this.titulo = titulo;
        this.isbn = isbn;
        this.anio = anio;
        this.disponible = disponible;
    }

    @Override
    public String toString() {
        return "Libro{" +
            "id=" + id +
            ", titulo='" + titulo + '\'' +
            ", isbn='" + isbn + '\'' +
            ", anio=" + anio +
            ", disponible=" + disponible +
            '}';
    }
}
```

¿Por qué `Integer` y no `int` en `anio`?

Porque `anio` en la base de datos puede ser `NULL`. Un `int` no puede ser `null`, pero `Integer` sí.

6.5 DAO: ejecutar SELECT y mapear ResultSet

DAO significa Data Access Object (Objeto de Acceso a Datos). Crea un paquete dao y dentro una clase LibroDao. Su trabajo es hablar con la base de datos y devolver objetos Java.

```
package com.dam.ad.dao;

import com.dam.ad.db.Db;
import com.dam.ad.model.Libro;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.ArrayList;
import java.util.List;

public class LibroDao {

    public List<Libro> findAll() {
        String sql = "SELECT id, titulo, isbn, anio, disponible FROM libro
ORDER BY id";

        try (Connection con = Db.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {

            List<Libro> libros = new ArrayList<>();

            while (rs.next()) {
                int id = rs.getInt("id");
                String titulo = rs.getString("titulo");
                String isbn = rs.getString("isbn");
                Integer anio = (Integer) rs.getObject("anio"); // admite NULL
                boolean disponible = rs.getBoolean("disponible");

                libros.add(new Libro(id, titulo, isbn, anio, disponible));
            }

            return libros;

        } catch (Exception e) {
            throw new RuntimeException("Error leyendo libros", e);
        }
    }
}
```

Qué debes entender aquí:

Con `public List<Libro> findAll()` declaramos un método al que llamamos `findAll` (o el nombre que elijas. En DAOs suele usarse `findAll` como “traer todos los registros”) que no recibe parámetros y devuelve una lista de objetos `Libro`.

Con `String sql = "SELECT id, titulo, isbn, anio, disponible FROM libro ORDER BY id";` estamos preparando el commando SELECT en un string llamado `sql` que posteriormente le pasaremos a la BD para que nos devuelva esas columnas/atributos.

Dentro del `try(-with-resources)` se crean y se cierran automáticamente tres recursos:

- `Connection con = Db.getConnection()` → abre la conexión.
- `PreparedStatement ps = con.prepareStatement(sql)` → prepara la consulta que teníamos en el string `sql` anterior para esa conexión `con`.
- `ResultSet rs = ps.executeQuery()` → ejecuta el SELECT y devuelve las filas de libros con todos los atributos que le pedimos en el string `sql`.

La ventaja es que al salir del Bloque `try`, `rs/ps/con` se cierran solos (evita fugas de conexiones).

6.6 Main: ejecutar la prueba

Crea un paquete `app` y dentro una clase `Main`. El main solo llama al DAO e imprime resultados.

```
package com.dam.ad.app;

import com.dam.ad.dao.LibroDao;

public class Main {
    public static void main(String[] args) {
        var dao = new LibroDao();
        System.out.println("=== LIBROS ===");
        dao.findAll().forEach(System.out::println);
    }
}
```

RECUERDA: Si imprime 3 libros, objetivo cumplido

Si no imprime nada, revisa primero el SELECT en `pgAdmin` (¿hay datos?), y luego revisa `db.properties`.

7. Problemas típicos y cómo resolverlos

Cuando algo falla, sigue este orden:

4. ¿El SELECT funciona en pgAdmin? (Si no, el problema es de BD, no de Java).
5. ¿db.properties apunta a la BD correcta (biblioteca_ad) y al puerto correcto (5432)?
6. ¿Usuario/password correctos?
7. ¿Maven descargó el driver? (Reload)

7.1 Tabla de errores frecuentes

Mensaje	Causa probable + solución
No suitable driver found	No está el driver JDBC o Maven no lo descargó. Revisa pom.xml y recarga Maven.
Connection refused	PostgreSQL apagado o puerto incorrecto. Arranca el servicio y revisa 5432.
password authentication failed	Password incorrecto. Revisa db.properties.
database "biblioteca_ad" does not exist	Nombre de BD mal en db.url. Revisa el nombre exacto (usar minúsculas).
No se encuentra db.properties	El archivo no está en src/main/resources o tiene otro nombre. Revisa ruta y recarga proyecto.